

DATA STRUCTURES · CASE STUDY

Browser Forward Navigation: Choosing the Right Data Structure

Why a stack is the natural fit for the Back and Forward buttons you use every day



A short study on browser history using stacks

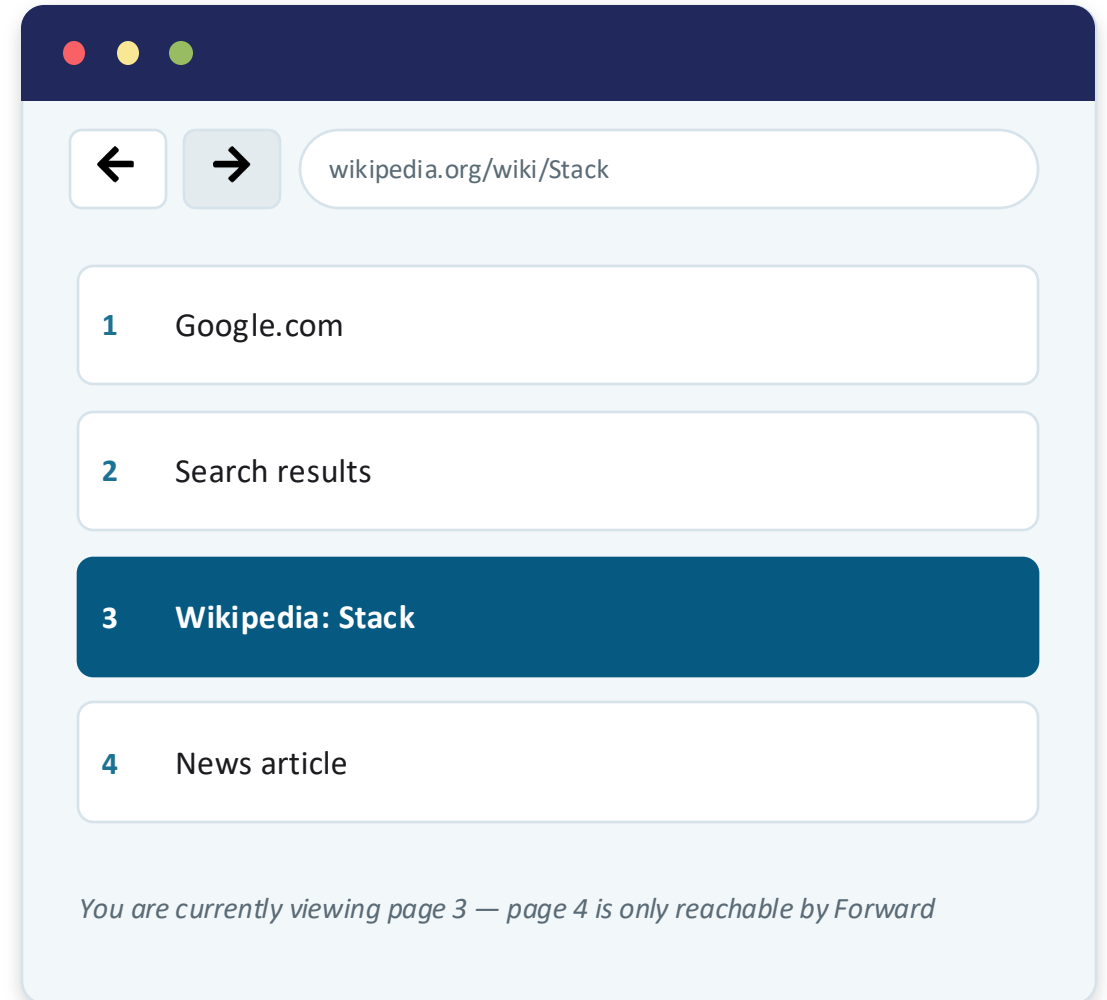
Sk. Ashwaq Ahamed
Reg no: 192572070

What happens when you click Back and Forward?

Think about browsing the internet: you open Google, search something, click a link to Wikipedia, then follow another link to a news article. At every point you can press Back to return, and once you go Back, a Forward button appears to redo that step.

The browser needs to:

- Remember every page you've visited, in order
- Undo the last visit instantly when you click Back
- Redo that exact visit if you click Forward next
- Forget the old Forward path the moment you visit a new page



THE RECOMMENDATION

Use a Stack — because history is Last In, First Out



A stack is a collection where the last item added is the first one removed — exactly like a stack of plates.

The last page you visited **is the first one you return to** when you press Back — that is exactly LIFO behaviour.

push ()

Visiting a new page adds it to the top

pop ()

Pressing Back removes the top page

peek ()

The browser shows you the top page instantly

News article

TOP

Wikipedia: Stack

Search results

Google.com



Back = remove
from top

HOW IT ACTUALLY WORKS

Two stacks working together

A single stack explains Back, but Forward needs a second stack to hold what you stepped away from.

BACK STACK (older pages)

Wikipedia: Stack

Search results

Google.com

CURRENTLY VIEWING

Wikipedia: Stack

New page → push current page onto Back Stack, clear Forward Stack

Click Back → pop Back Stack, push current page onto Forward Stack

Click Forward → pop Forward Stack, push current page onto Back Stack

FORWARD STACK (undone pages)

News article

Following one real browsing session



Why not an array, queue, or plain list?

Structure	Matches Back/Forward?	Reason
Queue (FIFO)	No	Would return your oldest page first, not your most recent one
Plain array (no order rule)	Partially	Works, but needs manual index tracking and offers no built-in undo logic
Linked list	Partially	Can act like a stack, but adds pointer overhead a stack doesn't need
Stack (LIFO)	Yes	push/pop directly mirror "visit a page" and "go back a page" — O(1) every time

CONCLUSION

Two stacks give browsers fast, correct navigation



A Back Stack and a Forward Stack model exactly how browsing history behaves



Every visit, Back, and Forward is a push or pop — $O(1)$ time, no searching needed



Visiting a fresh page correctly clears the Forward Stack, matching real browser behaviour

In short: whenever "undo the most recent action" is the rule, a stack is the right tool.